RegisterKaro

ENGINEERING HIRING · TAKE-HOME ASSIGNMENT

ITR Credentials Generation Automation

Build a bot that generates Income-Tax portal login credentials, stream every step as live events, and surface every run on an operations dashboard.

Automation | Live Events | Dashboard

CONFIDENTIAL — FOR CANDIDATE USE ONLY

POSITION
Full Stack Engineer

EFFORT
1 day only

STACK
Playwright · Node · Next.js

1 The Mandate

- Automate **generating and recovering Income-Tax e-filing portal login credentials** — a PAN-based User ID + password, gated by a CAPTCHA and an OTP.
- It must be **run by a bot**, and we want to **watch it happen in real time**.
- Build **one service** that: **generates** the credentials via browser automation; **streams** every step as live events to an operations UI; **saves the generated credentials** to the database; and **lists** every run on an admin dashboard.

> **Three things matter — and only these three:**
>
> 1. **Automation** — a bot that generates the credentials.
> 2. **Live events** — every step streamed `automation → server → UI` in real time, with replay.
> 3. **Dashboard** — operators watch a run live; admins see them all.

2 The Three Pillars

Automation

A Playwright bot that drives the live IT portal to generate a credential for a PAN — through a CAPTCHA and a human-supplied OTP.

YOU DESIGN THE FLOW

Live Events

Every step streamed automation → server → UI in real time, over SSE/WS, with replay on reconnect. The centrepiece.

NO GAPS · NO DUPES

Dashboard

A live console for one run and an admin table for all runs — live-updating, drill-down, masked PAN, metrics.

OPERATOR-READABLE

Pillar 1 · The Automation

- **Playwright (Chromium)** bot that drives the live Income-Tax portal (§6) to generate a login credential for a given PAN. Headless-capable, runnable headed for the demo.
- The on-portal flow passes through a **CAPTCHA gate** and an **OTP step**. **Model the run as an explicit state machine — that design is yours.** We are not prescribing the steps; we will judge the model you choose.
- The OTP step is **human-in-the-loop**: the bot must **pause**, signal that it is waiting, and **resume** when the OTP is supplied from the UI.
- Emit a **structured event** for every meaningful step (level, phase, step, message, timestamp) — not free-text `console.log`.
- **Mask secrets** (PAN / OTP / password) in every event. **Clean up the browser** on every exit. **One run failing must never touch another.**
- **On success, save the generated credentials to MongoDB** as the run's result (User ID + password) — stored securely (encrypted at rest, never written to logs), with a timestamp.

Pillar 2 · **Live Events** — the core of this assignment

Use a **webhook + SSE** pipeline (it mirrors how we run automations in production). The exact design is yours:

- **Bot → server over a webhook** — an authenticated POST per event / phase change; retries on failure so no event is lost.
- **Server** persists every event and is the single source of truth.
- **Server → browser over SSE** — the server streams events with an id per event.
- **Replay on (re)connect:** a viewer joining mid-run or after a drop gets the backlog, then the live tail — **no gaps, no duplicates**.
- **Bounded memory:** ring buffer for live fan-out + the durable Mongo log as replay source — never an unbounded array.
- **Not a poll:** a 1-second `GET` loop dressed up as "live" fails this pillar.

Pillar 3 · **The Dashboard**

- **Live console** — open a run, watch its events stream in (newest at the bottom, auto-scroll with a pause toggle), see the **current phase on a stepper**, and **supply the OTP** when the run asks for it. Survives a network blip and replays what it missed.
- **Admin dashboard** — every run in a table: job id, **masked PAN**, current phase, started / updated, duration, outcome. **Live-updating without a manual refresh.** Filter by phase / outcome; click a row to open that run's console + full history.
- **Metrics strip** — total runs, success rate, failures, p50 / p99 duration (from `/metrics`).
- **Minimal, professional, orange/white.** Readable by a non-technical operator; colour log lines by level only where it carries meaning (error = red, warn = amber). No rainbow UI.

3 Service Requirements

- A small, **layered** HTTP service owns the jobs and the event pipeline.
- **Node.js** preferred; justify any alternative.
- Persist job + event history in **MongoDB**.
- Expose what the dashboard needs: start a run, list all runs, one run's status, the live event stream + history, supply OTP, cancel, and basic metrics. **The route design is yours.**
- **Webhook ingest:** the bot pushes events to the service over an authenticated endpoint; the service persists and fans them out over SSE (per Pillar 2).
- **Auth:** a shared bearer token on mutating routes and a webhook secret on the event-ingest route; health-check exempt. Don't over-build it; don't leave it open.
- **Validation:** reject a bad `POST /jobs` (PAN shape, required fields) with a clear 400.
- **Correlation:** accept / echo `X-Request-Id` so a run is grep-traceable automation → service → UI.
- **Fail-soft:** a thrown error in one run returns a clean error for that run and never takes down the process.

4 Tech Stack

Pillar	Expected	Allowed alternative
Automation	Playwright + Chromium	Puppeteer (justify)
Service	Node.js	Any, if justified; must be a real HTTP service
Live events	Webhook (in) + SSE (out)	WebSocket out (justify)
Persistence	MongoDB	Another store only if justified
Dashboard	Next.js 15 + React + Phosphor	Any modern web framework

Pick boring, robust tools. We value clarity over cleverness.

5 The Engineering Bar (we grade the code, not just the demo)

- Write it as if it is going into our codebase on Monday.
- We read the repository like a pull request — **structure, modularity, and the database matter as much as the feature working**.

· Production-grade & modular

- **Clear layering, separation of concerns.** Transport (routes) ≠ domain (job lifecycle / state machine) ≠ persistence (a Mongo repository layer) ≠ automation. **No business logic in route handlers; no god-files.**

- **Shared shapes.** One definition of the event and job shape, reused across automation, service, and UI — not redeclared three times.
- **Config via environment** (no secrets, URLs, or tokens hard-coded), structured application logging, and a **graceful shutdown** that drains SSE connections and closes the Mongo client.
- **Errors handled, not swallowed.** Small, named, single-responsibility modules a reviewer can read top-to-bottom.

MongoDB modelling & optimization — we will read your schema, indexes, and queries

- **Deliberate data model:** jobs and their event stream modelled so both "list all runs" and "replay one run's events from a cursor" are cheap. Justify embedding vs. a separate `events` collection in `ARCHITECTURE.md`.
- **Indexes that match the queries** you actually run — e.g. `{ jobId, seq }` for event replay, an index for the admin list's sort/filter (phase, updatedAt). No collection scans on the hot paths.
- **Efficient access:** projections (don't fetch full documents to render a row), **cursor / range pagination** (not `skip` over large sets), and **bulk/append-efficient writes** for the event firehose. No N+1 round-trips to build the dashboard.

6 The Target Portal

- Run the automation against the **live Income-Tax e-filing portal**: `https://www.incometax.gov.in/iec/foportal/`. No mock — drive the real site.
- Model the real flow (identity → CAPTCHA → OTP → set password → confirmation) as your state machine.
- Handle the real conditions: bad / unreadable CAPTCHA, delayed OTP, wrong OTP, portal errors / timeouts.
- **Test with a PAN you control.** Keep real values in local env, mask them in every event, never commit or log PII.

7 Deliverables

1. **A Git repo** — `automation/`, `service/`, `ui/` (or a sensible layout). Commit in small, readable steps; we read the history.
2. **A** `README.md` a teammate can follow to run everything in <10 minutes: prerequisites, env vars, how to start each layer, how to trigger a run.
3. **An** `ARCHITECTURE.md` (1–2 pages): your state model, the event schema, how live streaming + replay works, what you persisted and why, and your trade-offs.
4. **A screen recording (5–8 min):** start a run from the UI, watch events stream live, supply the OTP, reach success; then show the admin dashboard + metrics; then trigger one **failure** (e.g. wrong OTP ×3) and show how it surfaces.
5. **Tests** for what deserves them: state-machine transitions, event replay-from-cursor, payload validation. Quality over quantity.

8 Ground Rules & Submission

- ✓ Run on the live portal (§6); test with a PAN you control — **never commit or log real PII**.
- ✓ AI assistance is fine; you must **defend every line** in the follow-up call.
- ✓ Ambiguous requirement? Make a call, note it in `ARCHITECTURE.md`, move on.
- ✓ Submit the **repo link + recording** to your point of contact.
- ✓ In the interview you'll walk us through the architecture and **extend it live** (e.g. add a state, or switch SSE↔WebSocket).
- ✓ Optional stretch: two runs streaming concurrently; a stale-run sweeper; resume mid-OTP after restart.

> **Build it like it ships Monday.** We care less about feature count and more about whether the automation is observable, fails safely, and could be handed to an operations team that has never met you.